МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Львівський національний університет імені Івана Франка

Факультет електроніки та комп'ютерних технологій

ЗВІТ

про виконання лабораторної роботи №4

«Професійне ООП та система типів Python»

з дисципліни «Професійний Python»

Варіант №5 — «Каталог автомобілів»

Виконав:
студент групи ФЕП-32
Горин Михайло
Репозиторій: github.com/MykhailoG212/Prof_python

Львів — 2026

2. Номер і тема лабораторної роботи

Лабораторна робота №4. Тема: «Професійне ООП та система типів Python». Звіт продовжує формат попередніх лабораторних: постановка задачі, таблиця відповідності вимог, опис реалізації, результати виконання, фрагменти коду та висновок.

3. Мета роботи

Сформувати практичні навички проєктування типізованої об'єктної моделі з dataclass, properties, успадкуванням, композицією, ABC, Protocol, generic classes, TypeVar, TypedDict і dependency injection. Застосувати принципи SOLID, відокремити application service від сховища та перевірити код через mypy.

4. Номер індивідуального варіанта

Варіант №5 — «Каталог автомобілів». ЛР4 є наступним етапом наскрізного проєкту після ЛР1–ЛР3: збережено каталог, аналітику, потоковий CSV pipeline та доповнено їх професійною domain model.

5. Постановка задачі

Потрібно реалізувати типізовану об'єктну модель каталогу автомобілів для сутностей Vehicle, Car, ElectricCar, VehicleCatalog і VehicleSpecification. Програма має додавати авто, шукати за маркою/моделлю, фільтрувати за роком, знаходити найдорожче авто, визначати середню ціну й найменший пробіг. Залежності repository та criteria потрібно описати через abstraction.

5.1. Вимоги індивідуального варіанта №5

Вимога варіанта	Реалізація
Domain objects Vehicle, Car, ElectricCar	models.py: базовий Vehicle, конкретний Car та ElectricCar із запасом ходу.
VehicleCatalog і VehicleSpecification	repositories.py: collection facade; models.py: frozen specification value object.
Додавання автомобіля	VehicleCatalog.add() та VehicleCatalogService.add().
Пошук за маркою / моделлю	BrandModelCriteria, VehicleCatalogService.search_by_brand_or_model().
Фільтрація за роком	YearCriteria та VehicleCatalogService.filter_by_year().

Вимога варіанта	Реалізація
Найдорожчий автомобіль	VehicleCatalogService.most_expensive().
Середня ціна й мінімальний пробіг	VehicleCatalogService.average_price() та lowest_mileage().
Repository та abstraction criteria	Repository[T] ABC; VehicleRepository та SearchCriteria[T] Protocol.

5.2. Загальні вимоги лабораторної роботи

Компонент	Де реалізовано
Не менше 5 domain classes	Vehicle, Car, ElectricCar, VehicleSpecification, VehicleCatalog; також application service.
Dataclass і frozen value object	Dataclass-моделі у models.py та criteria.py; VehicleSpecification має frozen=True.
Property та інкапсуляція	Vehicle.full_name; валідація у __post_init__; repository зберігає _items приватно.
Inheritance та composition	ElectricCar extends Car; VehicleCatalog містить Repository[Vehicle].
ABC і Protocol	Repository[T] є ABC; VehicleRepository та SearchCriteria[T] є Protocol.
Щонайменше 3 dunder methods	__str__, __len__, __iter__, __contains__, __getitem__.
DI та service layer	VehicleCatalogService отримує VehicleRepository через конструктор.
SOLID	SRP, OCP, LSP, ISP і DIP описані на прикладах цього проєкту.
TypeVar, Generic, TypedDict	Repository[T], T_contra у критерії, VehicleData у dto.py.
Static checker і main demo	mypy strict mode; пункт 13 main.py та окрема команда oop_demo.

6. Структура проєкту

Нові модулі відокремлюють domain model, критерії пошуку, контракти сховища, generic repository, DTO й application service. Зміни до main.py додають лише пункт меню для запуску демо; попередні можливості ЛР1–ЛР3 лишаються доступними.

```
src/car_catalog/
├── models.py          # Vehicle, Car, ElectricCar, VehicleSpecification
├── criteria.py        # BrandModelCriteria, YearCriteria
├── protocols.py       # SearchCriteria[T], VehicleRepository
├── repositories.py    # Repository[T], InMemoryRepository[T], VehicleCatalog
├── dto.py             # VehicleData (TypedDict)
├── application.py     # VehicleCatalogService
├── oop_demo.py        # composition root і сценарій демонстрації
└── main.py            # спільне меню; пункт 13 запускає демо ЛР4
```

Наявний клас Car зберігає попередній п’ятиаргументний виклик brand, model, year, price, mileage; specification має immutable default value. Таким чином поточні сервіси та звіти ЛР1–ЛР3 не потребують переписування.

7. Domain model і діаграма класів

Vehicle — базова dataclass-сутність із перевіркою спільних атрибутів. Car наслідує її стан і поведінку; ElectricCar додає range_km. VehicleSpecification зберігається як value object. Repository[T] — окремий справді абстрактний контракт сховища; VehicleCatalog композиційно використовує цей контракт.

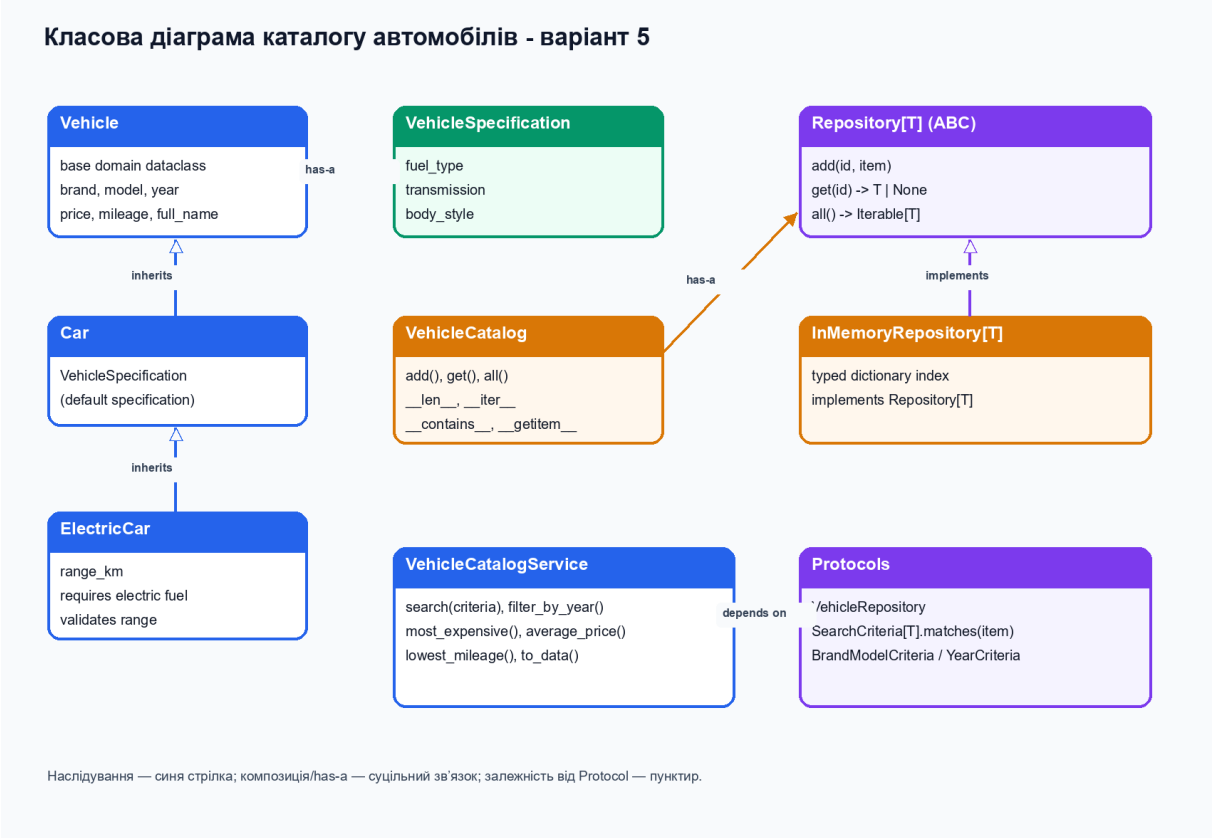


Рисунок 1 — Діаграма класів каталогу автомобілів, варіант 5

Синя стрілка показує inheritance: ElectricCar є Car, а Car є Vehicle. Суцільний зв’язок із ромбом позначає композицію: Vehicle має VehicleSpecification, а VehicleCatalog працює через Repository[Vehicle]. InMemoryRepository[T] реалізує ABC repository; application service залежить від Protocols.

Сутність	Відповідальність
Vehicle	Спільний валідований стан, full_name property та user-facing __str__.
Car / ElectricCar	Автомобіль загального типу / електричний підтип із перевіреним запасом ходу.
VehicleSpecification	Незмінні тип пального, трансмісія та тип кузова.
VehicleCatalog	Колекційний API з add/get/all та dunder-методами; має Repository[Vehicle].
VehicleCatalogService	Use cases: пошук, відбір за роком, max/min, середня ціна та DTO.
Criteria	BrandModelCriteria і YearCriteria інкапсулюють правила matches().

8. Поля та правила валідації

Поле	Тип / правило	Де перевіряється
brand, model	str; обидва значення непорожні	Vehicle.__post_init__()
year	int; не раніше 1886	Vehicle.__post_init__()
price	float; скінченна та невід'ємна	math.isfinite() і перевірка межі
mileage	int; невід'ємне	Vehicle.__post_init__()
VehicleSpecification	усі три рядки непорожні	VehicleSpecification.__post_init__()
ElectricCar.range_km	додатне число; пальне має бути електричним	ElectricCar.__post_init__() після super()

Frozen dataclass не дає непомітно змінити збережені поля після створення. Це робить VehicleSpecification value object, а Car і ElectricCar — стабільними записами каталогу.

9. Інкапсуляція, property і dunder methods

Метод	Результат
Vehicle.full_name	Повертає brand + model як обчислювану властивість.
Vehicle.__str__	Форматує назву, рік і ціну для користувацького виводу.
VehicleCatalog.__len__	Підтримує len(catalog).
VehicleCatalog.__iter__	Підтримує перебір через for.
VehicleCatalog.__contains__	Перевіряє ID конструкцією id in catalog.
VehicleCatalog.__getitem__	Дозволяє catalog[id] і піднімає KeyError для відсутнього ID.

Репозиторій приховує словник _items. Запис проходить перевірку ID у add(), а об'єкт автомобіля перевіряє власний стан у конструкторі. Публічний API повертає типізовані об'єкти, а не відкриває внутрішній mutable dict.

10. Наслідування та композиція

ElectricCar is a Car: він підтримує контракт Car, успадковує загальну валідацію через super() і додає лише специфічні для електроприводу правила. Це реальна subtype relationship, а не використання inheritance для повторного коду.

VehicleCatalog has a Repository: колекція не є репозиторієм і не наслідує конкретне сховище; вона делегує його операції об'єкту, переданому в конструктор. Композицію простіше замінити тестовим або файловим адаптером.

Підхід	Приклад	Вплив
Inheritance	ElectricCar → Car → Vehicle	Повторне використання спільного контракту підтипу.
Composition	VehicleCatalog → Repository[Vehicle]	Зберігання можна замінити без зміни доменної моделі.
Dependency injection	VehicleCatalogService(repository)	Зовнішня залежність передається ззовні, її можна підмінити у тесті.

11. Abstract Base Class та Protocol

Repository[T] успадковує ABC і містить abstractmethod add(), get() та all(). InMemoryRepository[T] реалізує їх через словник. Неповну реалізацію такого repository не можна створити як звичайний екземпляр.

SearchCriteria[T_contra] — runtime_checkable Protocol з одним методом matches(item). BrandModelCriteria і YearCriteria реалізують його структурно, без явного inheritance. VehicleRepository — вузький Protocol для сервісу: add, get і all.

ABC: Repository[T]	Protocol: criteria та repository API
Номінальний контракт із abstractmethod.	Структурна сумісність за наявністю потрібних методів.

ABC: Repository[T]	Protocol: criteria та repository API
Перевіряє повноту реалізації адаптера сховища.	Сервіс не залежить від конкретного класу реалізації.
Concrete implementation — InMemoryRepository[T].	Перевіряється runtime через @runtime_checkable та isinstance.

12. Generic repository, TypeVar та TypedDict

TypeVar T зв'язує тип елемента між add(), get() і all() у Repository[T]. Той самий generic repository можна параметризувати Car або іншим domain type. TypeVar T_contra є контраваріантним параметром SearchCriteria, оскільки критерій споживає об'єкт для перевірки.

VehicleData є TypedDict із ключами id, brand, model, year, price, mileage та kind. Runtime representation лишається звичайним словником; dataclass використовується для доменних сутностей, TypedDict — для API/JSON-подібного payload.

```
T = TypeVar("T")

class Repository(ABC, Generic[T]):
    @abstractmethod
    def add(self, item_id: int, item: T) -> None: ...

    @abstractmethod
    def get(self, item_id: int) -> T | None: ...

    @abstractmethod
    def all(self) -> Iterable[T]: ...
```

13. Application service, dependency injection і SOLID

VehicleCatalogService отримує VehicleRepository у конструкторі. oop_demo.py є composition root: створює InMemoryRepository[Vehicle], передає його сервісу, додає демонстраційні записи й виконує use cases. Бізнес-логіка працює через abstraction, а не створює сховище самостійно.

Принцип	Де видно у коді
SRP	Repository зберігає дані; criteria визначають відбір; service виконує сценарії; DTO задає зовнішню форму.
OCP	Новий search strategy реалізує matches(), без редагування циклу пошуку у service.
LSP	ElectricCar замінює Car і зберігає базовий контракт та валідацію.

Принцип	Де видно у коді
ISP	VehicleRepository і SearchCriteria є невеликими спеціалізованими Protocols.
DIP	VehicleCatalogService залежить від VehicleRepository, а не від InMemoryRepository.

Ін’єкція залежності дає змогу передати реалізацію для пам’яті, майбутнє файлове сховище або тестовий fake. Сервіс можна перевіряти незалежно від способу зберігання.

14. Демонстраційний запуск

Демо додає Toyota Camry, BMW M3 і Tesla Model 3. Воно перевіряє пошук за маркою, фільтр за роком, найдорожчий запис, середню ціну та авто з мінімальним пробігом.

```
python -m car_catalog.oop_demo
# або python -m car_catalog.main та вибрати пункт 13
```



Рисунок 2 — Фактичний вивід демонстраційного сценарію ЛР4

За наведеними даними середня ціна становить (25 000 + 72 000 + 41 000) / 3 = 46 000 USD. Найдорожче авто — BMW M3 за 72 000 USD; найменший пробіг — у Tesla Model 3, 12 000 км.

15. Автоматизоване тестування

До наявного test suite додано tests/test_oop_model.py із вісьмома перевірками domain model, immutable specification, валідації ElectricCar, generic repository, collection dunder methods, runtime Protocol, application service, порожнього каталогу й TypedDict DTO.

Набір	Що перевіряється	Кількість
tests/test_oop_model.py	ЛР4: моделі, критерії, repository, dunder methods, сервіс і DTO.	8 нових тестів

Набір	Що перевіряється	Кількість
tests/test_main.py, test_services.py, test_processors_analytics.py	Регресія інтерфейсу та можливостей ЛР1–ЛР2.	20 тестів
tests/test_streaming.py	Ітератори, CSV pipeline, batching і benchmark ЛР3.	20 тестів
Увесь suite	Запуск усіх наявних та нових тестів pytest.	48 passed, 0 failed

```
python -m pytest -v --basetemp=report/_pytest_run
```



Рисунок 3 — Фактичний результат повного прогону pytest

Фінальний запуск завершився результатом 48 passed за 4.36 секунди. Тимчасова папка pytest була розміщена в межах проєкту, щоб обійти обмеження системної тимчасової директорії цього середовища.

16. Статична перевірка типів

Для ЛР4 використано mypy у strict mode. Налаштування цільових семи модулів міститься в pyproject.toml; пакет має py.typed, щоб позначити доступність type hints.

```
python -m pip install -e ".[dev]"
python -m mypy
Success: no issues found in 7 source files
```

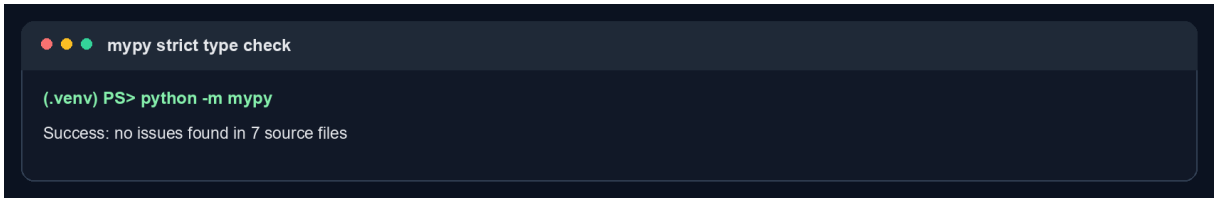


Рисунок 4 — Фактичний результат mypy strict mode

16.1. Type-checking experiment

У тимчасовий файл було внесено три умисні помилки: рядок присвоюється int, рядок повертається з функції з int return type, а str

передається generic repository, параметризованому Car. Перевірка виконувалася mypy --strict; тестовий файл після експерименту видалено.

```
customer_id: int = "1"

def total() -> int:
    return "100"

repository: InMemoryRepository[Car] = InMemoryRepository()
repository.add(1, "not a car")
```

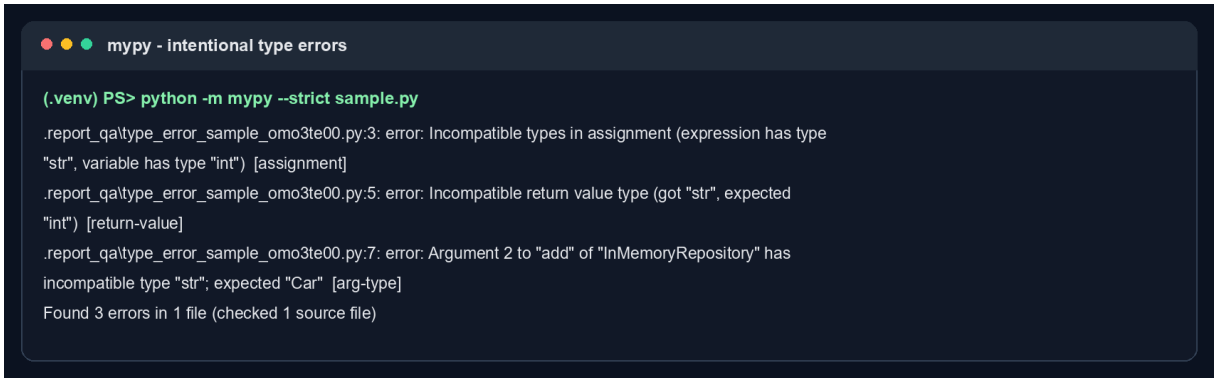


Рисунок 5 — Diagnostics mypy для трьох навмисних type errors

Mypy повідомив про incompatible assignment (str замість int), incompatible return value (str замість int) і incompatible argument для add() (str замість Car). Виправлення: призначити 1, повернути 100 і передати Car(...). Після виправлення production-код проходить повну налаштовану перевірку.

17. Завдання підвищеної складності

Додатково реалізовано такі можливості з переліку завдань підвищеної складності:

Advanced feature	Реалізація у проєкті
Parameterized generic repository	Repository[T] та InMemoryRepository[T] зі збереженням типу сутності.
Contravariant TypeVar	T_contra у SearchCriteria[T_contra] відповідає позиції споживання параметра.
runtime_checkable Protocol	SearchCriteria перевіряється через isinstance; є окрема перевірка у pytest.
Custom collection class	VehicleCatalog реалізує __getitem__, __iter__, __contains__ та __len__.
Strategy pattern	BrandModelCriteria та YearCriteria замінюють умовні гілки в сервісі.
Immutable value objects	VehicleSpecification і записи автомобілів оголошені frozen dataclass.

Advanced feature	Реалізація у проєкті
Repository abstraction та DI	Сервіс приймає VehicleRepository Protocol; concrete storage обирається composition root.
Тестовий та статичний контроль	48 passing tests, 8 із них перевіряють ООП-модель; strict mypy clean run.

Ці можливості виходять за межі мінімальної реалізації: вони перевіряють розширюваність типів, runtime-сумісність Protocol, поведінку custom collection і заміну інфраструктури без зміни сервісу.

18. Порівняння дизайну залежностей

Варіант	Схема	Наслідок
A: concrete dependency	Service усередині створює InMemoryRepository.	Сильне зчеплення; тест і заміна сховища вимагають редагування service.
B: DI через abstraction	Service(repository: VehicleRepository).	Сховище замінюється зовні; бізнес-логіка ізольована і тестована.

Реалізація відповідає варіанту B. Наслідування використане лише для реального відношення підтипу автомобіля, а змінні інфраструктурні залежності підключені композицією.

19. Відповіді на контрольні запитання

Що таке class та object? Клас описує структуру й поведінку; object є екземпляром класу.

Що таке instance attribute? Поле, значення якого належить конкретному екземпляру.

Що таке encapsulation? Контроль стану об’єкта через API та валідацію допустимих значень.

Для чого property? Для обчислюваного або контрольованого значення з атрибутоподібним доступом.

Що означає is-a / has-a? is-a означає subtype; has-a означає композиційну складову.

Коли composition краща за inheritance? Коли об’єкт використовує незалежний компонент, який треба замінювати.

Для чого super()? Для виклику реалізації базового класу в ієрархії.

Що таке polymorphism? Різні об’єкти підтримують спільну операцію за одним контрактом.

Що таке ABC? Базовий клас із абстрактними операціями, які concrete subtype має реалізувати.

Що таке Protocol? Опис структурного інтерфейсу за набором методів та їх типами.

Чим Protocol відрізняється від ABC? Protocol допускає structural compatibility; ABC зазвичай використовує явне успадкування.

Що генерує dataclass? Зокрема __init__, __repr__ та __eq__ для полів класу.

Що означає frozen=True? Забороняє зміну полів після конструювання dataclass.

Для чого __post_init__()? Для валідації або додаткової ініціалізації після присвоєння полів.

Що таке dunder method? Спеціальний метод для інтеграції класу з Python protocols та операторами.

Що означає SRP? Клас має одну основну відповідальність і причину для зміни.

Що означає OCP? Нові варіанти поведінки додаються розширенням, а не зміною стабільного ядра.

Що означає LSP? Subtype має замінювати base type без порушення його контракту.

Що означають ISP та DIP? Інтерфейси вузькі; high-level логіка залежить від abstraction.

Що таке dependency injection? Передавання залежності в об'єкт ззовні, наприклад через constructor.

Чи перевіряє Python type hints автоматично? Ні; статичний аналіз виконує окремий інструмент.

Для чого mypy? Він перевіряє узгодженість типових анотацій і використання типів без запуску логіки.

Що таке TypeVar та Generic? TypeVar задає параметр типу, Generic дозволяє параметризувати клас цим типом.

Що таке TypedDict? Статичний опис структури словника з визначеними ключами.

Чим TypedDict відрізняється від dataclass? TypedDict лишається dict; dataclass створює доменний екземпляр класу.

Що довів type-checking experiment? Mypy знаходить несумісне присвоєння, return type та аргумент generic repository.

Який результат повного тестування? 48 тестів пройшли успішно, серед них 8 нових перевірок ЛР4.

20. Лістинги коду

Нижче наведено точний вміст файлів, що реалізують ООП-шар і його перевірки. Лістинги побудовано без ручного переписування, безпосередньо з актуальних файлів робочого проєкту.

models.py

Лістинг звірено з робочим файлом src/car_catalog/models.py. Нижче наведено його фактичний вміст.

```
"""Типізована об'єктна модель каталогу автомобілів (ЛР4)."""


import math
from abc import ABC
from dataclasses import dataclass



@dataclass(slots=True, frozen=True)
class VehicleSpecification:
    """Незмінні технічні характеристики транспортного засобу."""


    fuel_type: str
    transmission: str
    body_style: str


    def __post_init__(self) -> None:
        if not all(value.strip() for value in (self.fuel_type, self.transmission, self.body_style)):
            raise ValueError("Характеристики транспортного засобу не можуть бути порожніми")
```

```
@dataclass(slots=True, frozen=True)
class Vehicle(ABC):
    """Базова сутність каталогу; конкретний транспорт має ціну та характеристики."""

    brand: str
    model: str
    year: int
    price: float
    mileage: int
    specification: VehicleSpecification

    def __post_init__(self) -> None:
        if not self.brand.strip() or not self.model.strip():
            raise ValueError("Марка та модель не можуть бути порожніми")
        if self.year < 1886:
            raise ValueError(f"Некоректний рік випуску: {self.year}")
        if not math.isfinite(self.price) or self.price < 0:
            raise ValueError("Ціна має бути скінченною та невід'ємною")
        if self.mileage < 0:
            raise ValueError("Пробіг не може бути від'ємним")

    @property
    def full_name(self) -> str:
        return f"{self.brand} {self.model}"

    def __str__(self) -> str:
        return f"{self.full_name} ({self.year}) — ${self.price:,.2f}"


@dataclass(slots=True, frozen=True)
class Car(Vehicle):
    """Модель даних автомобіля.

    Атрибути:
        brand: Марка автомобіля.
        model: Модель автомобіля.
        year: Рік випуску (>= 1886).
        price: Ціна в USD (>= 0).
        mileage: Пробіг у кілометрах (>= 0).
    """

    specification: VehicleSpecification = VehicleSpecification("Бензин", "Автоматична", "Седан")


@dataclass(slots=True, frozen=True)
class ElectricCar(Car):
    """Автомобіль з електричним приводом; запас ходу задається в кілометрах."""

    range_km: int = 0

    def __post_init__(self) -> None:
        super(ElectricCar, self).__post_init__()
        if self.range_km <= 0:
            raise ValueError("Запас ходу електромобіля має бути додатним")
        if self.specification.fuel_type.casefold() not in {"електро", "electric"}:
            raise ValueError("Для ElectricCar тип пального має бути 'Електро'")
```

criteria.py

Лістинг звірено з робочим файлом src/car_catalog/criteria.py. Нижче наведено його фактичний вміст.

```
"""Реалізації критеріїв пошуку каталогу."""
```

```
from dataclasses import dataclass


from car_catalog.models import Vehicle



@dataclass(frozen=True, slots=True)
class BrandModelCriteria:
    brand: str | None = None
    model: str | None = None


    def matches(self, item: Vehicle, /) -> bool:
        return (
            (self.brand is None or item.brand.casefold() == self.brand.strip().casefold())
            and (self.model is None or item.model.casefold() == self.model.strip().casefold())
        )



@dataclass(frozen=True, slots=True)
class YearCriteria:
    year: int


    def matches(self, item: Vehicle, /) -> bool:
        return item.year == self.year
```

protocols.py

Лістинг звірено з робочим файлом src/car_catalog/protocols.py. Нижче наведено його фактичний вміст.

```
"""Абстракції для пошуку та збереження каталогу."""


from collections.abc import Iterable
from typing import Protocol, TypeVar, runtime_checkable


from car_catalog.models import Vehicle


T_contra = TypeVar("T_contra", contravariant=True)



@runtime_checkable
class SearchCriteria(Protocol[T_contra]):
    """Structural interface критерію пошуку."""


    def matches(self, item: T_contra, /) -> bool: ...



class VehicleRepository(Protocol):
    """Мінімальний інтерфейс сховища, потрібний сервісу каталогу."""


    def add(self, item_id: int, item: Vehicle) -> None: ...


    def get(self, item_id: int) -> Vehicle | None: ...


    def all(self) -> Iterable[Vehicle]: ...
```

repositories.py

Лістинг звірено з робочим файлом src/car_catalog/repositories.py. Нижче наведено його фактичний вміст.

```python
"""Універсальне in-memory сховище та конкретний репозиторій авто."""


from abc import ABC, abstractmethod
from collections.abc import Iterable, Iterator
from typing import Generic, TypeVar


from car_catalog.models import Vehicle


T = TypeVar("T")



class Repository(ABC, Generic[T]):
    """Контракт типізованого сховища сутностей."""


    @abstractmethod
    def add(self, item_id: int, item: T) -> None: ...


    @abstractmethod
    def get(self, item_id: int) -> T | None: ...


    @abstractmethod
    def all(self) -> Iterable[T]: ...



class InMemoryRepository(Repository[T]):
    """Generic repository з інкапсульованим індексом за ID."""


    def __init__(self) -> None:
        self._items: dict[int, T] = {}


    def add(self, item_id: int, item: T) -> None:
        if item_id <= 0:
            raise ValueError("ID має бути додатним")
        self._items[item_id] = item


    def get(self, item_id: int) -> T | None:
        return self._items.get(item_id)


    def all(self) -> Iterable[T]:
        return tuple(self._items.values())



class VehicleCatalog:
    """Колекція транспортних засобів, що композиційно використовує repository."""


    def __init__(self, repository: Repository[Vehicle] | None = None) -> None:
        self._repository: Repository[Vehicle] = repository or InMemoryRepository[Vehicle]()
        self._next_id = 1


    def add(self, vehicle: Vehicle) -> int:
        item_id = self._next_id
        self._repository.add(item_id, vehicle)
        self._next_id += 1
        return item_id


    def get(self, item_id: int) -> Vehicle | None:
        return self._repository.get(item_id)


    def all(self) -> tuple[Vehicle, ...]:
        return tuple(self._repository.all())


    def __len__(self) -> int:
```

```
        return sum(1 for _ in self._repository.all())


    def __iter__(self) -> Iterator[Vehicle]:
        return iter(self._repository.all())


    def __contains__(self, item_id: object) -> bool:
        return isinstance(item_id, int) and self._repository.get(item_id) is not None


    def __getitem__(self, item_id: int) -> Vehicle:
        vehicle = self._repository.get(item_id)
        if vehicle is None:
            raise KeyError(item_id)
        return vehicle
```

dto.py

Лістинг звірено з робочим файлом src/car_catalog/dto.py. Нижче наведено його фактичний вміст.

```
"""TypedDict контракт для JSON-подібного представлення авто."""


from typing import TypedDict



class VehicleData(TypedDict):
    id: int
    brand: str
    model: str
    year: int
    price: float
    mileage: int
    kind: str
```

application.py

Лістинг звірено з робочим файлом src/car_catalog/application.py. Нижче наведено його фактичний вміст.

```
"""Application service: use cases каталогу з DI та залежністю від Protocol."""


from car_catalog.criteria import BrandModelCriteria, YearCriteria
from car_catalog.dto import VehicleData
from car_catalog.models import ElectricCar, Vehicle
from car_catalog.protocols import SearchCriteria, VehicleRepository



class VehicleCatalogService:
    """Виконує сценарії каталогу, отримуючи repository ззовні."""


    def __init__(self, repository: VehicleRepository) -> None:
        self._repository = repository
        self._next_id = 1


    def add(self, vehicle: Vehicle) -> int:
        item_id = self._next_id
        self._repository.add(item_id, vehicle)
        self._next_id += 1
        return item_id


    def search(self, criteria: SearchCriteria[Vehicle]) -> list[Vehicle]:
        return [vehicle for vehicle in self._repository.all() if criteria.matches(vehicle)]


    def search_by_brand_or_model(self, brand: str | None = None, model: str | None = None) ->
list[Vehicle]:
```

```
        return self.search(BrandModelCriteria(brand=brand, model=model))


    def filter_by_year(self, year: int) -> list[Vehicle]:
        return self.search(YearCriteria(year))


    def most_expensive(self) -> Vehicle | None:
        return max(self._repository.all(), key=lambda vehicle: vehicle.price, default=None)


    def average_price(self) -> float:
        vehicles = tuple(self._repository.all())
        return sum(vehicle.price for vehicle in vehicles) / len(vehicles) if vehicles else 0.0


    def lowest_mileage(self) -> Vehicle | None:
        return min(self._repository.all(), key=lambda vehicle: vehicle.mileage, default=None)


    def to_data(self, item_id: int, vehicle: Vehicle) -> VehicleData:
        return {
            "id": item_id,
            "brand": vehicle.brand,
            "model": vehicle.model,
            "year": vehicle.year,
            "price": vehicle.price,
            "mileage": vehicle.mileage,
            "kind": "electric" if isinstance(vehicle, ElectricCar) else "car",
        }
```

oop_demo.py

Лістинг звірено з робочим файлом src/car_catalog/oop_demo.py. Нижче наведено його фактичний вміст.

```
"""Демонстрація об'єктної моделі ЛР4 для варіанта 5."""


from car_catalog.application import VehicleCatalogService
from car_catalog.models import Car, ElectricCar, Vehicle, VehicleSpecification
from car_catalog.repositories import InMemoryRepository



def run_demo() -> None:
    repository: InMemoryRepository[Vehicle] = InMemoryRepository()
    service = VehicleCatalogService(repository)
    service.add(Car("Toyota", "Camry", 2020, 25_000.0, 45_000))
    service.add(Car("BMW", "M3", 2022, 72_000.0, 18_000))
    service.add(
        ElectricCar(
            "Tesla", "Model 3", 2023, 41_000.0, 12_000,
            VehicleSpecification("Електро", "Автоматична", "Седан"), 513,
        )
    )


    print("\n--- ООП-модель каталогу (ЛР4, варіант 5) ---")
    print("Пошук Toyota:", [str(car) for car in service.search_by_brand_or_model(brand="Toyota")])
    print("Авто 2023 року:", [str(car) for car in service.filter_by_year(2023)])
    print("Найдорожче авто:", service.most_expensive())
    print(f"Середня ціна: ${service.average_price():,.2f}")
    print("Мінімальний пробіг:", service.lowest_mileage())



if __name__ == "__main__":
    run_demo()
```

20.1. Тестовий файл ЛР4

Вісім тестів перевіряють стійкість моделі й інтеграцію application layer.

```
from dataclasses import FrozenInstanceError
```

```
import pytest


from car_catalog.application import VehicleCatalogService
from car_catalog.criteria import BrandModelCriteria, YearCriteria
from car_catalog.dto import VehicleData
from car_catalog.models import Car, ElectricCar, Vehicle, VehicleSpecification
from car_catalog.protocols import SearchCriteria
from car_catalog.repositories import InMemoryRepository, VehicleCatalog



def electric_car() -> ElectricCar:
    return ElectricCar(
        "Tesla",
        "Model 3",
        2023,
        41_000.0,
        12 000,
        VehicleSpecification("Електро", "Автоматична", "Седан"),
        513,
    )



def test_vehicle_hierarchy_and_frozen_specification() -> None:
    car = Car("Toyota", "Camry", 2020, 25_000.0, 45_000)
    electric = electric_car()


    assert isinstance(car, Vehicle)
    assert isinstance(electric, Car)
    assert car.full_name == "Toyota Camry"
    assert str(car) == "Toyota Camry (2020) — $25,000.00"
    with pytest.raises(FrozenInstanceError):
        electric.specification.fuel_type = "Бензин"  # type: ignore[misc]



def test_vehicle_and_specification_validation() -> None:
    with pytest.raises(ValueError, match="не можуть бути порожніми"):
        VehicleSpecification("", "Автоматична", "Седан")
    with pytest.raises(ValueError, match="1800"):
        Car("Toyota", "Oldtimer", 1800, 1_000.0, 0)
    with pytest.raises(ValueError, match="скінченною"):
        Car("Toyota", "Camry", 2020, float("inf"), 0)



def test_electric_car_requires_valid_range_and_electric_specification() -> None:
    with pytest.raises(ValueError, match="додатним"):
        ElectricCar(
            "Tesla", "Model 3", 2023, 41_000.0, 12_000,
            VehicleSpecification("Електро", "Автоматична", "Седан"), 0,
        )
    with pytest.raises(ValueError, match="тип пального"):
        ElectricCar("Tesla", "Model 3", 2023, 41_000.0, 12_000, range_km=513)



def test_generic_repository_validates_ids_and_returns_entries() -> None:
    repository: InMemoryRepository[Car] = InMemoryRepository()
    car = Car("Toyota", "Camry", 2020, 25_000.0, 45_000)


    with pytest.raises(ValueError, match="додатним"):
        repository.add(0, car)
    repository.add(1, car)
    assert repository.get(1) == car
    assert repository.get(2) is None
    assert list(repository.all()) == [car]
```

```
def test_vehicle_catalog_collection_dunder_methods() -> None:
    catalog = VehicleCatalog()
    first = Car("Toyota", "Camry", 2020, 25_000.0, 45_000)
    second = electric_car()
    first_id = catalog.add(first)
    second_id = catalog.add(second)


    assert (first_id, second_id) == (1, 2)
    assert len(catalog) == 2
    assert list(catalog) == [first, second]
    assert 1 in catalog
    assert 3 not in catalog
    assert catalog[2] == second
    with pytest.raises(KeyError):
        _ = catalog[3]



def test_criteria_are_structurally_compatible_and_match_case_insensitively() -> None:
    vehicle = electric_car()
    criteria: SearchCriteria[Vehicle] = BrandModelCriteria("tesla", "MODEL 3")


    assert isinstance(criteria, SearchCriteria)
    assert criteria.matches(vehicle)
    assert not BrandModelCriteria("Tesla", "Roadster").matches(vehicle)
    assert YearCriteria(2023).matches(vehicle)
    assert not YearCriteria(2022).matches(vehicle)



def test_application_service_adds_searches_and_calculates_statistics() -> None:
    repository: InMemoryRepository[Vehicle] = InMemoryRepository()
    service = VehicleCatalogService(repository)
    cars: list[Vehicle] = [
        Car("Toyota", "Camry", 2020, 25_000.0, 45_000),
        Car("BMW", "M3", 2022, 72_000.0, 18_000),
        electric_car(),
    ]


    assert [service.add(car) for car in cars] == [1, 2, 3]
    assert service.search_by_brand_or_model(brand="TOYOTA") == [cars[0]]
    assert service.search_by_brand_or_model(model="Model 3") == [cars[2]]
    assert service.filter_by_year(2023) == [cars[2]]
    assert service.most_expensive() == cars[1]
    assert service.average_price() == pytest.approx(46_000.0)
    assert service.lowest_mileage() == cars[2]



def test_application_service_handles_empty_repository_and_typed_dto() -> None:
    repository: InMemoryRepository[Vehicle] = InMemoryRepository()
    service = VehicleCatalogService(repository)


    assert service.most_expensive() is None
    assert service.lowest_mileage() is None
    assert service.average_price() == 0.0


    car = electric car()
    data: VehicleData = service.to_data(7, car)
    assert data == {
        "id": 7,
        "brand": "Tesla",
        "model": "Model 3",
        "year": 2023,
        "price": 41_000.0,
        "mileage": 12_000,
        "kind": "electric",
    }
```

21. Висновок

У лабораторній роботі №4 для варіанта 5 типізовано й розширено каталог автомобілів. Реалізовано Vehicle, Car, ElectricCar, VehicleSpecification, VehicleCatalog, generic repository, критерії пошуку та application service із dependency injection.

Діаграма відображає реальні inheritance, composition та залежності від інтерфейсів. Додано frozen value object, property, custom collection dunder methods, ABC, runtime-checkable Protocol, generic types і TypedDict. SRP, OCP, LSP, ISP та DIP проілюстровано на компонентах проєкту.

Вісім нових pytest-перевірок доповнили наявний suite; повний запуск показав 48 passed. Муру strict mode знаходить описані навмисні помилки й завершує перевірку робочого коду результатом Success: no issues found in 7 source files.